https://share.google/aimode/RhIXeglRJxYabT9Cv

To set up multi-tenancy with independent schemas in Django, the most common and robust tool is django-tenants, which leverages PostgreSQL schemas to isolate tenant data while sharing a single application instance. [1, 2]

1. Installation

Install the package along with the PostgreSQL database adapter:

pip install django-tenants

2. Configure `settings.py` [3]

You must separate your applications into those shared globally (like the tenant manager) and those specific to individual tenants. [1, 4]

- **DATABASE_ENGINE**: Update to the specialized PostgreSQL backend.

```
DATABASES = {
    'default': {
        'ENGINE': 'django_tenants.postgresql_backend',
        'NAME': 'your_db_name',
        # ... other credentials
    }
}
```

-
- **SHARED_APPS & TENANT_APPS**: Split your installed apps.

```
SHARED_APPS = (
    'django_tenants',  # Must be first
    'customers',       # Where you'll define the Tenant model
    'django.contrib.contenttypes',
    'django.contrib.auth',
    # ... global apps
)

TENANT_APPS = (
    'django.contrib.contenttypes',
    'app_with_tenant_data',
    # ... apps that need isolated data
)
```

```
INSTALLED_APPS = list(SHARED_APPS) + [app for app in TENANT_APPS if app not in SHARED_APPS]
```

-
- **Middleware & Router**: Add the tenant middleware to the top of the list so it can identify the tenant before processing views.

```
MIDDLEWARE = [
    'django_tenants.middleware.main.TenantMainMiddleware',
    # ... other middleware
]

DATABASE_ROUTERS = (
    'django_tenants.routers.TenantSyncRouter',
)

TENANT_MODEL = 'customers.Client'  # Define your tenant model path
TENANT_DOMAIN_MODEL = 'customers.Domain'
```

-
- [1, 4, 5]

3. Create Tenant Models

In your shared `customers` app, define the models that represent a tenant and its associated domain. [6, 7]

```
from django.db import models
from django_tenants.models import TenantMixin, DomainMixin

class Client(TenantMixin):
    name = models.CharField(max_length=100)
    # default true, so that when a tenant is saved, a schema is created
    auto_create_schema = True

class Domain(DomainMixin):
    pass
```

4. Run Migrations

Use specialized commands to sync schemas instead of the standard `migrate`. [2, 4]

1. **Shared Migrations**: `python manage.py migrate_schemas --shared`
2. **Tenant Migrations**: `python manage.py migrate_schemas --tenant` [8, 9]

5. Create Your First Tenant

You can create tenants through the Django shell or a custom management command. Every tenant requires a "public" entry first to handle the main site/domain. [1, 4, 10, 11, 12]

```
from customers.models import Client, Domain

# Create the public tenant
tenant = Client(schema_name='public', name='Schemas Inc.')
tenant.save()

# Add a domain to the public tenant
domain = Domain(domain='localhost', tenant=tenant, is_primary=True)
domain.save()

# Create an actual customer tenant
customer = Client(schema_name='tenant1', name='First Client')
customer.save()

domain = Domain(domain='tenant1.localhost', tenant=customer, is_primary=True)
domain.save()
```

When a request comes to `tenant1.localhost`, the middleware automatically switches the PostgreSQL `search_path` to the `tenant1` schema, isolating all subsequent ORM queries. [1, 11]

Are you planning to use **subdomains** for identification, or do you need to identify tenants via **request headers**? [13, 14, 15]

6. Create Your Another Tenant

To create another tenant in a `django-tenants` setup, you can use the interactive management command or create one programmatically through the Django shell or a view. [1, 2]

1. Using Management Commands

The easiest way to create a tenant manually is through the terminal. This command will prompt you for the required fields defined in your tenant model (e.g., name, schema name). [1, 3]

```
python manage.py create_tenant
```

If you need to create a superuser for that specific tenant schema immediately after, use: [1, 4]

```
python manage.py create_tenant_superuser --schema=tenant_schema_name
```

2. Programmatic Creation (Python Shell or Views) [5]

Creating a tenant works like saving any other Django model. When you call `.save()`, the package automatically creates the PostgreSQL schema and runs migrations for all apps listed in `TENANT_APPS`. [1, 6, 7]

```
from customers.models import Client, Domain

# 1. Create the tenant object
tenant = Client(
    schema_name='brand_new_tenant',
    name='Brand New Client'
)
tenant.save()  # This triggers schema creation and migrations

# 2. Add a domain so the middleware can route requests
domain = Domain(
    domain='tenant2.localhost',
    tenant=tenant,
    is_primary=True
)
domain.save()
```

3. Key Requirements for New Tenants

- **Unique Schema Name**: Every tenant must have a unique `schema_name` that follows PostgreSQL naming conventions (e.g., no hyphens, starts with a letter).
- **Domain Mapping**: A tenant is only accessible if a corresponding `Domain` entry exists. The middleware uses the request's hostname to look up the tenant in the public schema.

- **Public Tenant**: Ensure you have already created a "public" tenant (usually with `schema_name='public'`) to handle the main site and shared resources before creating additional private tenants. [2, 6, 7, 8, 9]

Hidden operational and architectural details

1. The Migration "Multiplier" Effect

When you run `migrate_schemas --tenant`, Django runs migrations for **every** schema sequentially. [2, 3]

- **The Issue**: If you have 10 tenants, migrations take seconds. If you have 500, it could take 30+ minutes.
- **Best Practice**: Plan for zero-downtime migrations and ensure your CI/CD pipeline can handle long-running database operations. Avoid making large numbers of schemas on a single instance if your table structure changes frequently. [4, 5]

2. User Authentication & Cross-Tenant Users

By default, `django-tenants` doesn't handle the logic of "who can access which tenant."

- **The Trap**: A user created in the `public` schema exists globally, but they may only be authorized for specific tenants.
- **The Solution**: Most developers use django-tenant-users to manage memberships, cross-tenant permissions, and invite flows. [6, 7]

3. Shared vs. Isolated Relationships

Establishing relationships between a `SHARED_APP` model and a `TENANT_APP` model is tricky. [6]

- **Foreign Key Constraint**: A model in the `public` schema cannot easily have a foreign key to a model in a tenant schema because that table doesn't exist globally.
- **Deletion Errors**: Deleting a global user that is linked to a tenant model can trigger `on_delete` cascades that fail because the database can't find the tenant's table from the public context. Use `DO_NOTHING` or careful manual cleanups for these fields. [8]

4. Background Tasks (Celery)

Standard background workers are often "tenant-blind."

- **The Risk**: If a Celery task is triggered from a tenant view, the worker might try to execute it in the `public` schema, causing `Table does not exist` errors or data leaks.
- **Requirement**: You must pass the `schema_name` to your tasks and use `django_tenants.utils.schema_context` to wrap the task logic.

```
from django_tenants.utils import schema_context


@shared_task
def process_data(schema_name, data_id):
    with schema_context(schema_name):
        # Task logic now runs in the correct schema
        ...
```

-
- [4, 9]

5. Static & Media File Isolation

By default, media files are often stored in a single folder.

- **Security Risk**: Without proper configuration, `tenant1` might be able to guess the URL for `tenant2`'s private uploads.
- **Solution**: Configure a `DEFAULT_FILE_STORAGE` that includes the tenant name or ID in the file path to keep assets logically separated at the storage level. [10]

6. The "Noisy Neighbor" Problem

A single large tenant running heavy reports can slow down the database for all other tenants because they share the same CPU, RAM, and connection pool. [9, 11]

- **Monitoring**: Implement per-tenant resource monitoring early so you can identify which client is hogging resources before it crashes the site.

Example

This demonstration project uses django-tenants to isolate data via **PostgreSQL schemas**. In this scenario, we will build a "Tenant Task Manager" where different companies (tenants) can manage their own private list of tasks using the same codebase.

Project Architecture

- **Shared App (`customers`)**: Lives in the `public` schema. It stores the registry of all tenants and their domains.
- **Tenant App (`tasks`)**: Lives in individual tenant schemas. It stores task data unique to each company.

Step 1: Install and Initialize

Install the necessary packages and start your project.

```
pip install django-tenants psycopg2-binary
django-admin startproject tenant_demo .
python manage.py startapp customers  # Shared app
python manage.py startapp tasks      # Tenant app
```

Step 2: Configure Settings

In `settings.py`, categorize your apps and update the database engine.

```
# settings.py
DATABASES = {
    'default': {
        'ENGINE': 'django_tenants.postgresql_backend', # Required
        'NAME': 'tenant_db',
        'USER': 'postgres',
        # ... other credentials
    }
}

SHARED_APPS = [
    'django_tenants',
    'customers',  # Our tenant manager
    'django.contrib.contenttypes',
```

```
    'django.contrib.auth',
    'django.contrib.sessions',
    'django.contrib.admin',
]

TENANT_APPS = [
    'tasks', # Data in this app will be isolated per schema
    'django.contrib.auth', # Allow separate users per tenant
]

INSTALLED_APPS = list(SHARED_APPS) + [app for app in TENANT_APPS if app not in
SHARED_APPS]

TENANT_MODEL = 'customers.Client'
TENANT_DOMAIN_MODEL = 'customers.Domain'

MIDDLEWARE = [
    'django_tenants.middleware.main.TenantMainMiddleware', # Must be first
    # ... other middleware
]

DATABASE_ROUTERS = ('django_tenants.routers.TenantSyncRouter',) # Required
```

Step 3: Define Models

In your `customers` app, define the tenant registry.

```
# customers/models.py
from django.db import models
from django_tenants.models import TenantMixin, DomainMixin

class Client(TenantMixin):
    name = models.CharField(max_length=100)
    created_on = models.DateField(auto_now_add=True)

class Domain(DomainMixin):
    pass
```

In your `tasks` app, define the tenant-specific data.

```
# tasks/models.py
from django.db import models
```

```
class Task(models.Model):
    title = models.CharField(max_length=200)
    is_completed = models.BooleanField(default=False)
```

Step 4: Run Migrations and Create Tenants

Use the `migrate_schemas` command to set up the database structure.

1. **Migrate Public Schema**: `python manage.py migrate_schemas --shared`
2. **Create First Tenants** (via Python shell):

```
from customers.models import Client, Domain

# 1. The 'Public' tenant is mandatory for the main domain
public = Client.objects.create(schema_name='public', name='Main Site')
Domain.objects.create(domain='localhost', tenant=public, is_primary=True)

# 2. Create an actual customer tenant
client1 = Client.objects.create(schema_name='company_a', name='Company A')
Domain.objects.create(domain='a.localhost', tenant=client1, is_primary=True)
```

3.
4. **Migrate Tenant Schema**: `python manage.py migrate_schemas --tenant`

Step 5: Test Isolation

Run your server using `python manage.py runserver localhost:8000`.

- Access `http://a.localhost`: Any tasks created here will belong only to "Company A".
- If you create `b.localhost`, it will have a completely empty task list, demonstrating that the database tables are physically separate at the schema level.

Admin Interface

To customize the Django Admin interface to display the current tenant's name in the header, you can use one of two methods: a quick setting in `urls.py` or a more flexible template override. [1, 2, 3]

Method 1: Dynamic Site Header (Quickest)

Since `django-tenants` attaches the current tenant object to the request, you can dynamically set the admin header in your main `urls.py` file. This change is global and affects the "Django Administration" text at the top of every admin page. [1, 4]

```
# urls.py
from django.contrib import admin
from django.urls import path

# Function to dynamically set the header
def set_admin_header(request):
    # 'request.tenant' is added by TenantMainMiddleware
    tenant_name = getattr(request.tenant, 'name', 'Shared')
    admin.site.site_header = f"{tenant_name} Administration"
    admin.site.site_title = f"{tenant_name} Admin Portal"

# You can wrap the admin.site.urls or use a custom admin site class
# to ensure this runs on every request.
```

Method 2: Template Override (Most Flexible)

If you want the tenant name to appear alongside the default branding or in a specific layout, you should override the `base_site.html` template. [5, 6]

1. **Create the directory structure**: In your project root, create a folder named `templates/admin/`.
2. **Add `base_site.html`**: Create this file inside that folder.
3. **Use the `request.tenant` object**:

```
{% extends "admin/base_site.html" %}

{% block branding %}
<h1 id="site-name">
    <a href="{% url 'admin:index' %}">
        <!-- Access the name field from your Client model -->
```

```
      {{ request.tenant.name|default:"Django" }} Admin
    </a>
</h1>
{% endblock %}
```

4.
5. **Register the Template Path**: Ensure your `settings.py` includes this directory so Django looks there first.

```
TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [os.path.join(BASE_DIR, 'templates')], # Add this
        'APP_DIRS': True,
        # ...
    },
]
```

6.
7. [7, 8, 9]

Important Considerations

- **Request Availability**: For `request.tenant` to be available in templates, ensure `'django.template.context_processors.request'` is included in your `TEMPLATES` context processors.
- **Public vs. Private**: When logged into the "public" schema, `request.tenant.name` will reflect your main site name. In subdomains, it will automatically switch to that client's name. [9, 10, 11]

Sample Application

To give you a complete picture, here is a functional "Minified" multi-tenant project. This includes a shared tenant manager and an isolated "Invoices" app with full CRUD capabilities.

1. Project Structure

```
tenant_project/
├── customers/       # Shared (Global)
├── billing/         # Tenant-Specific (Isolated)
├── tenant_project/  # Core Settings
└── manage.py
```

2. Core Configuration (`settings.py`)

Add `django-tenants` and split your apps.

```
DATABASES = {
    'default': {
        'ENGINE': 'django_tenants.postgresql_backend',
        'NAME': 'multi_tenant_db',
        'USER': 'postgres',
        'PASSWORD': 'password',
        'HOST': 'localhost',
    }
}

SHARED_APPS = [
    'django_tenants',
    'customers',
    'django.contrib.contenttypes',
    'django.contrib.auth',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.admin',
]

TENANT_APPS = [
    'billing',  # The CRUD app
    'django.contrib.auth',
]
```

```
INSTALLED_APPS = list(SHARED_APPS) + [app for app in TENANT_APPS if app not in
SHARED_APPS]

TENANT_MODEL = 'customers.Client'
TENANT_DOMAIN_MODEL = 'customers.Domain'

MIDDLEWARE = [
    'django_tenants.middleware.main.TenantMainMiddleware',
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
]
```

3. The Models

Shared App (`customers/models.py`): Defines who the tenants are.

```
from django_tenants.models import TenantMixin, DomainMixin
from django.db import models

class Client(TenantMixin):
    name = models.CharField(max_length=100)
    auto_create_schema = True

class Domain(DomainMixin):
    pass
```

Tenant App (`billing/models.py`): The data we want to isolate.

```
from django.db import models

class Invoice(models.Model):
    client_name = models.CharField(max_length=100)
    amount = models.DecimalField(max_digits=10, decimal_places=2)
    description = models.TextField()

    def __str__(self):
        return f"{self.client_name} - ${self.amount}"
```

4. Tenant-Aware CRUD (`billing/views.py`)

Because the middleware switches the schema automatically based on the URL, these standard views **only** see data for the active tenant.

```
from django.shortcuts import render, redirect, get_object_or_404
from .models import Invoice

def invoice_list(request):
    # This automatically queries only the current tenant's schema
    invoices = Invoice.objects.all()
    return render(request, 'billing/list.html', {'invoices': invoices, 'tenant': request.tenant})

def invoice_create(request):
    if request.method == "POST":
        Invoice.objects.create(
            client_name=request.POST['client_name'],
            amount=request.POST['amount'],
            description=request.POST['description']
        )
        return redirect('invoice_list')
    return render(request, 'billing/form.html')

def invoice_delete(request, pk):
    get_object_or_404(Invoice, pk=pk).delete()
    return redirect('invoice_list')
```

5. URL Routing (`billing/urls.py`)

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.invoice_list, name='invoice_list'),
    path('new/', views.invoice_create, name='invoice_create'),
    path('delete/<int:pk>/', views.invoice_delete, name='invoice_delete'),
]
```

6. Templates (`billing/templates/billing/list.html`)

```
<h1>{{ tenant.name }}'s Invoices</h1>
<a href="{% url 'invoice_create' %}">Create New Invoice</a>
```

```
<ul>
  {% for invoice in invoices %}
  <li>{{ invoice.client_name }}: ${{ invoice.amount }}
    <a href="{% url 'invoice_delete' invoice.pk %}">Delete</a>
  </li>
  {% endfor %}
</ul>
```

7. Execution Commands

To see this in action:

1. **Sync everything**:
   `python manage.py migrate_schemas --shared`
2. **Create the public entry**:
   Run `python manage.py shell`:

```
from customers.models import Client, Domain
p = Client.objects.create(schema_name='public', name='Main System')
Domain.objects.create(domain='localhost', tenant=p, is_primary=True)
```

3.
4. **Create two different tenants**:

```
t1 = Client.objects.create(schema_name='apple', name='Apple Inc')
Domain.objects.create(domain='apple.localhost', tenant=t1, is_primary=True)

t2 = Client.objects.create(schema_name='google', name='Google LLC')
Domain.objects.create(domain='google.localhost', tenant=t2, is_primary=True)
```

5.
6. **Run Server**: `python manage.py runserver`
7. **Observe**: Go to `apple.localhost:8000`. Create an invoice. Then go to `google.localhost:8000`. The invoice list will be **empty**, even though it's the exact same code and database.

Do you need help configuring your **local hosts file** so your computer recognizes `apple.localhost` and `google.localhost`?